

Swinburne University of Technology
School of Science, Computing, and Emerging Technologies

SWE40006 - Software Deployment and Evolution
Deployment Portfolio Task 3

Student name: Ngoc Van Nhi Do
Student ID: 105551765
Submission date: DON'T FORGET THISSSSSSSSSSSS

Deployment Portfolio Task 3

1 Declared Target Level

All four subtasks 3.1, 3.2, 3.3, and 3.4 were attempted.

Table 1 below contain relevant links as proof of successful task execution.

Table 1: Publicly accessible URLs for live grading verification

Source	URL
Task 3.1	http://13.55.158.77

2 Execution Evidence

2.1 Task 3.1

For this task, I deployed a WordPress application onto an Amazon EC2 instance.

2.1.1 AWS Environment Setup

First, I created a new AWS account on the free plan, which gave me \$100 in free credits.

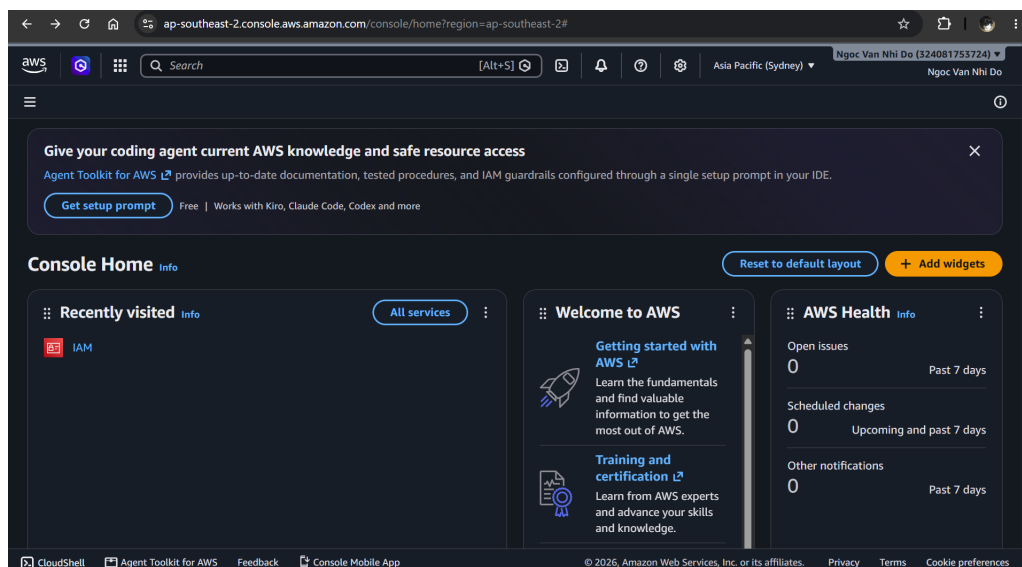


Figure 1: The AWS Console interface after I had successfully signed up.

I then created a new IAM user for my coursework named `SWE40006_student` rather than using the root account. This user is assigned to a new user group `EC2-Lab-Users` which is given full access to EC2 resources (using `AmazonEC2FullAccess`).

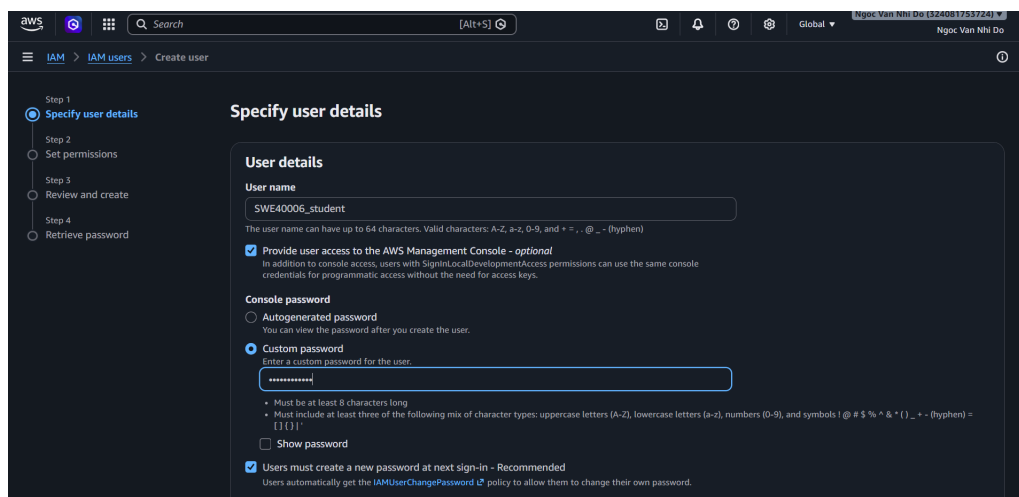


Figure 2: Creating a new IAM user SWE40006_student.

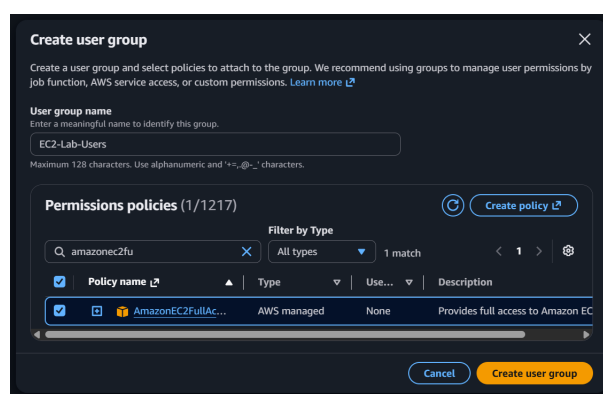


Figure 3: Adding AmazonEC2FullAccess to the user group EC2-Lab-Users, which is then assigned to SWE40006_student.

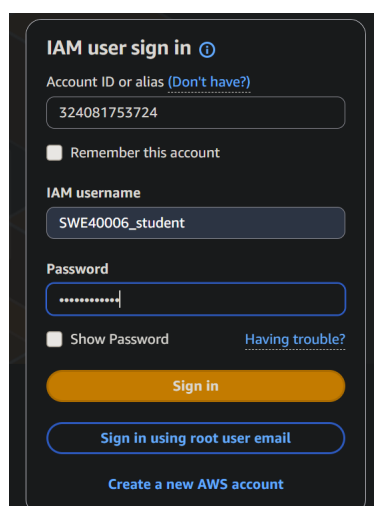
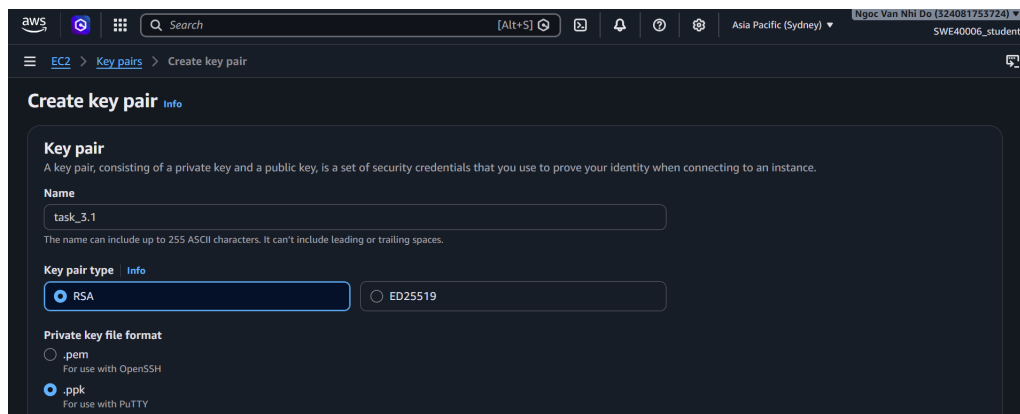


Figure 4: Logging into the AWS Console as the new IAM user.

To securely login into my EC2 instances, I must generate an SSH key pair. The private key was downloaded and stored on my local machine for later authentication.

Figure 5: Creating a new key pair `task-3.1`.

2.1.2 Compute Provisioning

I launched a new EC2 instance called `Task 3.1`. This instance runs Amazon Linux 2023, uses a `t3.micro` instance type to remain eligible for the free tier, and is configured with the previously generated key pair.

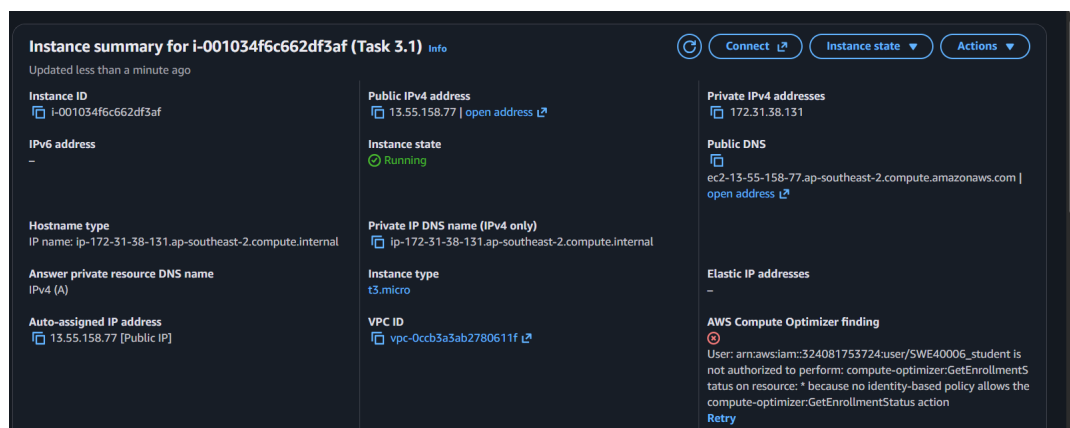


Figure 6: Creating a new Amazon EC2 instance.

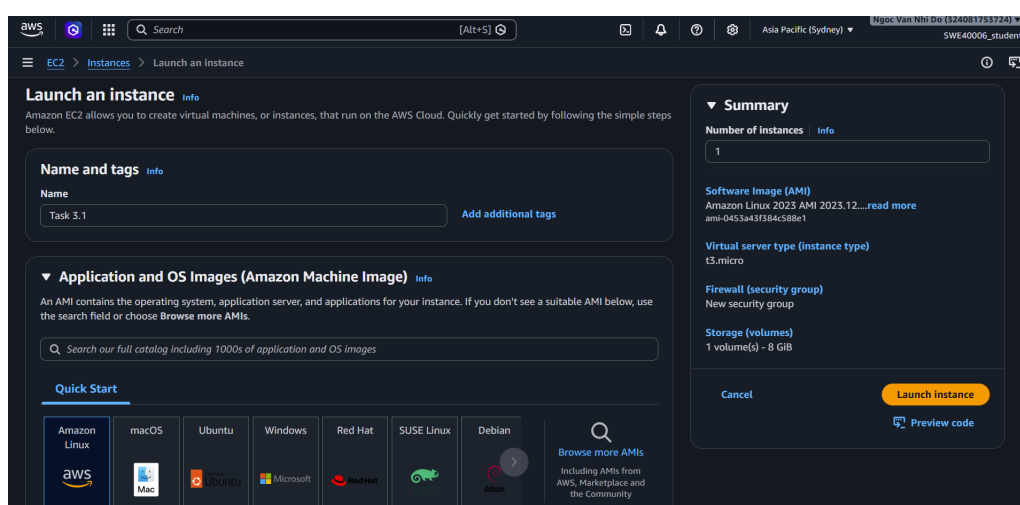
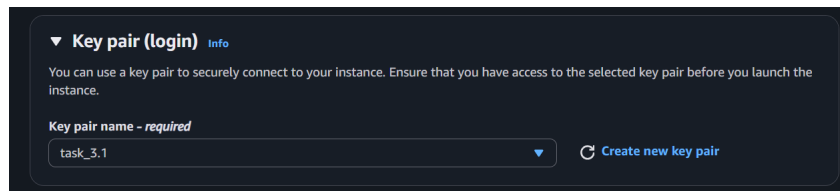


Figure 7: Amazon Linux 2023 was chosen as the OS for this instance.

Figure 8: Configuring the instance with the `task-3.1` key pair.

I then configured the instance with a new security group `task-3.1-sg`, which allowed SSH (22), HTTP (80), and HTTPS (443) inbound traffic from anywhere.

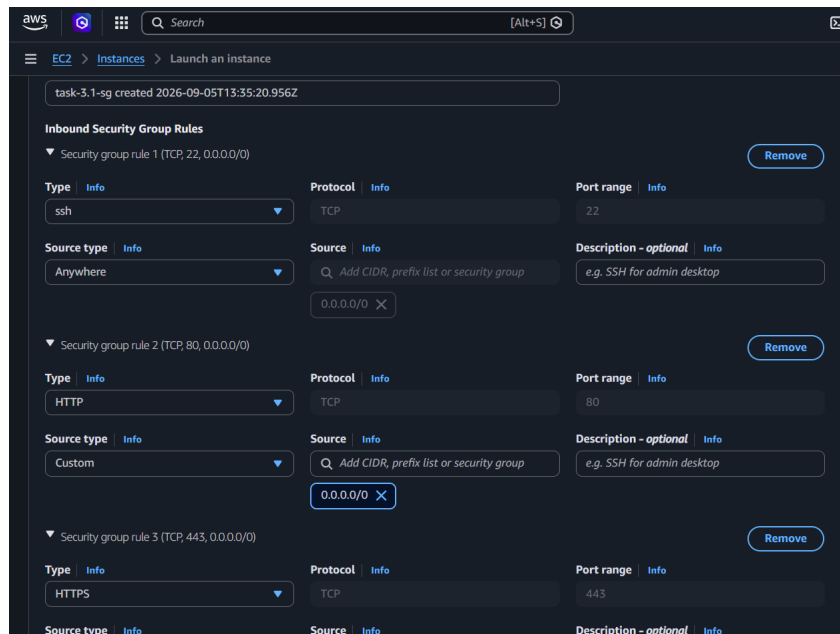


Figure 9: Configuring the security rules to allow inbound SSH, HTTP, and HTTPS traffic to this instance.

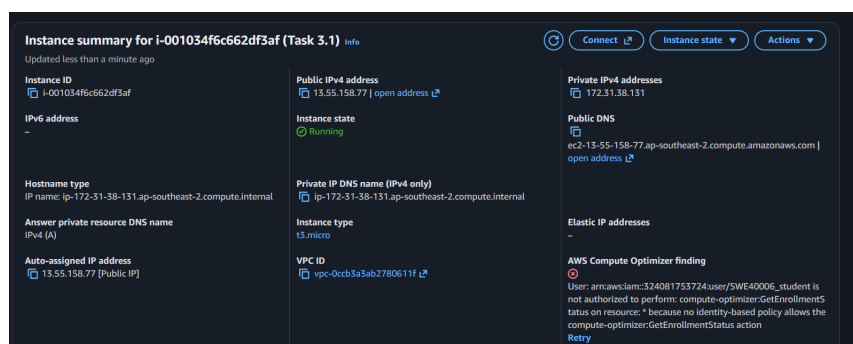


Figure 10: The new instance is successfully created.

I then SSH-ed to the instance using PuTTY.

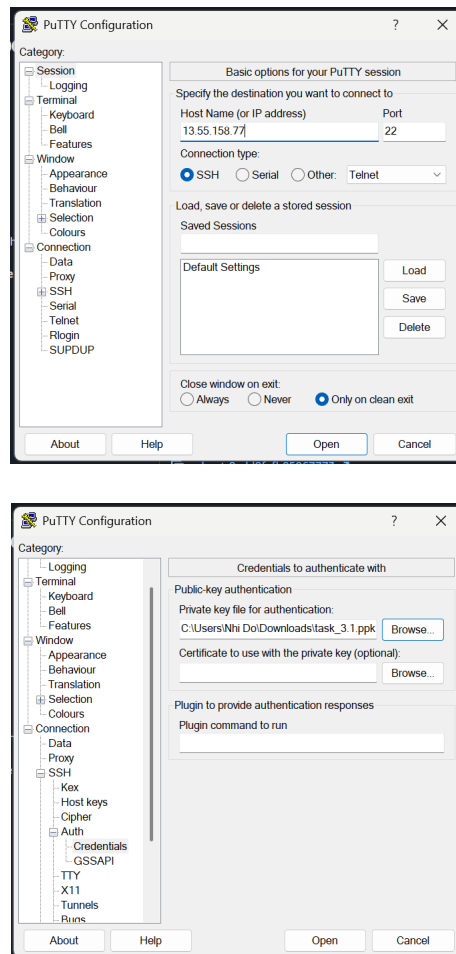
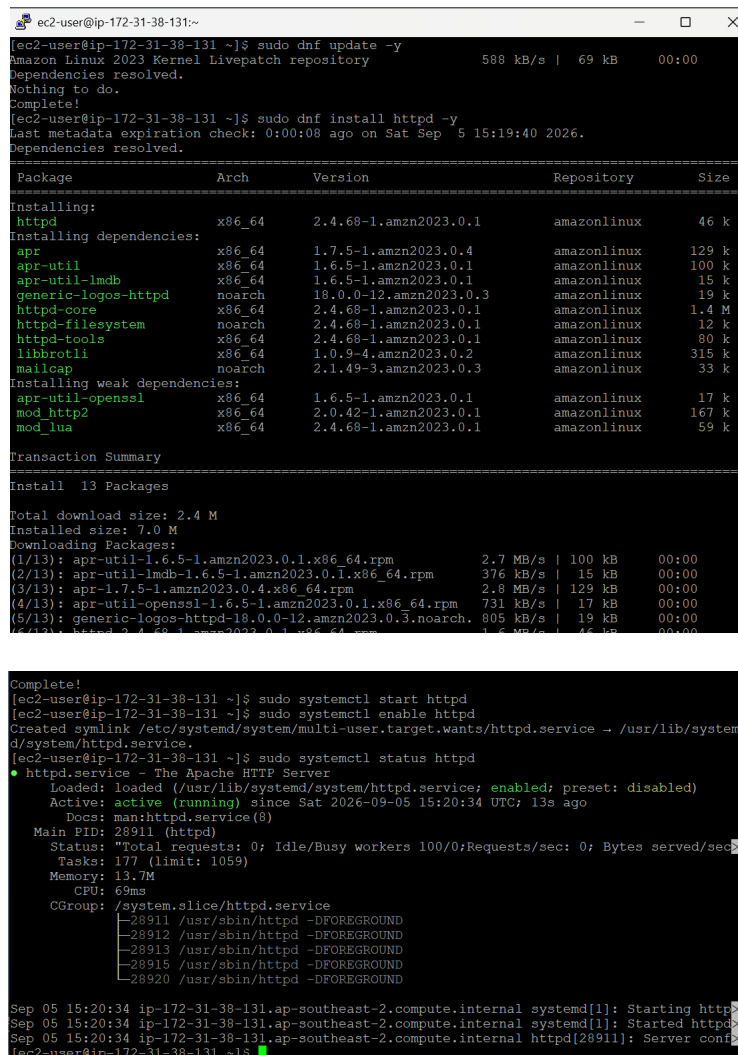


Figure 11: Connecting to the EC2 instance using SSH.



Figure 12: Connection was successful.

I then installed the Apache web server and started it so that it could later host my web application.



```

ec2-user@ip-172-31-38-131:~$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
ec2-user@ip-172-31-38-131:~$ sudo dnf install httpd -y
Last metadata expiration check: 0:00:08 ago on Sat Sep  5 15:19:40 2026.
Dependencies resolved.

=====
Package                Arch      Version                Repository              Size
=====
Installing:
httpd                  x86_64    2.4.68-1.amzn2023.0.1  amazonlinux              46 k
Installing dependencies:
apr                    x86_64    1.7.5-1.amzn2023.0.4   amazonlinux             129 k
apr-util               x86_64    1.6.5-1.amzn2023.0.1   amazonlinux             100 k
apr-util-ldap          x86_64    1.6.5-1.amzn2023.0.1   amazonlinux             15 k
generic-logos-httpd    noarch    18.0.0-12.amzn2023.0.3  amazonlinux             19 k
httpd-core              x86_64    2.4.68-1.amzn2023.0.1  amazonlinux            1.4 M
httpd-filesystem        noarch    2.4.68-1.amzn2023.0.1  amazonlinux             12 k
httpd-tools             x86_64    2.4.68-1.amzn2023.0.1  amazonlinux             80 k
libbrotli               x86_64    1.0.9-4.amzn2023.0.2   amazonlinux            315 k
mailcap                 noarch    2.1.49-3.amzn2023.0.3  amazonlinux             33 k
Installing weak dependencies:
apr-util-openssl        x86_64    1.6.5-1.amzn2023.0.1   amazonlinux             17 k
mod_http2               x86_64    2.0.42-1.amzn2023.0.1   amazonlinux            167 k
mod_lua                 x86_64    2.4.68-1.amzn2023.0.1   amazonlinux             59 k
=====
Transaction Summary
=====
Install 13 Packages

Total download size: 2.4 M
Installed size: 7.0 M
Downloading Packages:
(1/13): apr-util-1.6.5-1.amzn2023.0.1.x86_64.rpm      2.7 MB/s | 100 kB  00:00
(2/13): apr-util-ldap-1.6.5-1.amzn2023.0.1.x86_64.rpm 376 kB/s | 15 kB  00:00
(3/13): apr-1.7.5-1.amzn2023.0.4.x86_64.rpm          2.8 MB/s | 129 kB  00:00
(4/13): apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64.rpm 731 kB/s | 17 kB  00:00
(5/13): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch. 805 kB/s | 19 kB  00:00
(6/13): httpd-2.4.68-1.amzn2023.0.1.x86_64.rpm      1.5 MB/s | 46 kB  00:00
(7/13): httpd-core-2.4.68-1.amzn2023.0.1.x86_64.rpm 1.4 MB/s | 1.4 MB  00:00
(8/13): httpd-filesystem-2.4.68-1.amzn2023.0.1.noarch. 12 kB/s | 12 kB  00:00
(9/13): httpd-tools-2.4.68-1.amzn2023.0.1.x86_64.rpm 80 kB/s | 80 kB  00:00
(10/13): libbrotli-1.0.9-4.amzn2023.0.2.x86_64.rpm 315 kB/s | 315 kB  00:00
(11/13): mailcap-2.1.49-3.amzn2023.0.3.noarch.       33 kB/s | 33 kB  00:00
(12/13): mod_http2-2.0.42-1.amzn2023.0.1.x86_64.rpm 167 kB/s | 167 kB  00:00
(13/13): mod_lua-2.4.68-1.amzn2023.0.1.x86_64.rpm    59 kB/s | 59 kB  00:00
Complete!
ec2-user@ip-172-31-38-131:~$ sudo systemctl start httpd
ec2-user@ip-172-31-38-131:~$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
ec2-user@ip-172-31-38-131:~$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-09-05 15:20:34 UTC; 13s ago
     Docs: man:httpd.service(8)
   Main PID: 28911 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0"
   Tasks: 177 (limit: 1059)
  Memory: 13.7M
     CPU: 69ms
   CGroup: /system.slice/httpd.service
           └─ 28911 /usr/sbin/httpd -DFOREGROUND
              28912 /usr/sbin/httpd -DFOREGROUND
              28913 /usr/sbin/httpd -DFOREGROUND
              28915 /usr/sbin/httpd -DFOREGROUND
              28920 /usr/sbin/httpd -DFOREGROUND
Sep 05 15:20:34 ip-172-31-38-131.ap-southeast-2.compute.internal systemd[1]: Starting httpd: The Apache HTTP Server.
Sep 05 15:20:34 ip-172-31-38-131.ap-southeast-2.compute.internal systemd[1]: Started httpd: The Apache HTTP Server.
Sep 05 15:20:34 ip-172-31-38-131.ap-southeast-2.compute.internal httpd[28911]: Server configured.
ec2-user@ip-172-31-38-131:~$

```

Figure 13: Installing and starting the web server.

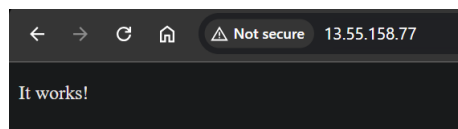


Figure 14: Verifying that the web server is publicly accessible.

2.1.3 Application Deployment

Before deploying the WordPress application, I needed to install its dependencies which included installing PHP and MariaDB.

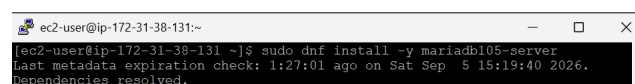


```

ec2-user@ip-172-31-38-131:~$ sudo dnf install php php-mysqlnd php-gd php-mbstring php-xml php-json php-curl php-zip -y
Last metadata expiration check: 1:23:32 ago on Sat Sep  5 15:19:40 2026.
Dependencies resolved.

```

Figure 15: Installing PHP.

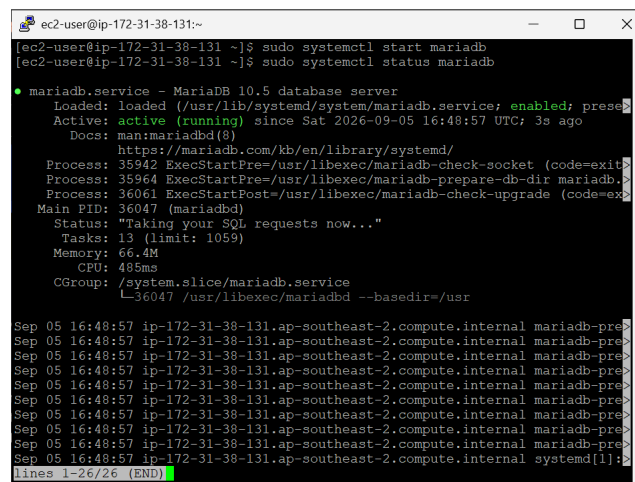


```

ec2-user@ip-172-31-38-131:~$ sudo dnf install -y mariadb105-server
Last metadata expiration check: 1:27:01 ago on Sat Sep  5 15:19:40 2026.
Dependencies resolved.

```

Figure 16: Installing MariaDB.



```

ec2-user@ip-172-31-38-131:~$ sudo systemctl start mariadb
ec2-user@ip-172-31-38-131:~$ sudo systemctl status mariadb

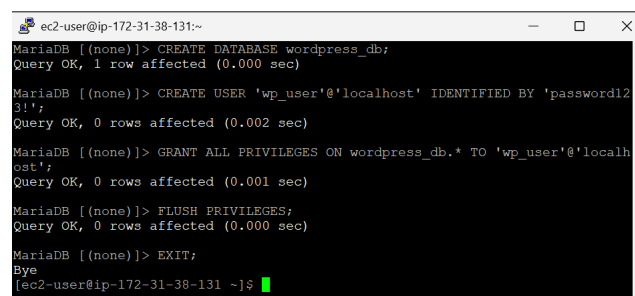
● mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; prese
   Active: active (running) since Sat 2026-09-05 16:48:57 UTC; 3s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 35942 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exit
   Process: 35964 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.>
   Process: 36061 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=ex
   Main PID: 36047 (mariadb)
    Status: "Taking your SQL requests now..."
      Tasks: 13 (limit: 1059)
     Memory: 66.4M
        CPU: 485ms
   CGroup: /system.slice/mariadb.service
           └─36047 /usr/libexec/mariadb --basedir=/usr

Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
Sep 05 16:48:57 ip-172-31-38-131.ap-southeast-2.compute.internal mariadb-pre>
lines 1-26/26 (END)

```

Figure 17: Starting and enabling the MariaDB service.

I then set up a local database for the web application.



```

MariaDB [(none)]> CREATE DATABASE wordpress_db;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> CREATE USER 'wp_user'@'localhost' IDENTIFIED BY 'password123!';
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_user'@'localhost';
Query OK, 0 rows affected (0.001 sec)

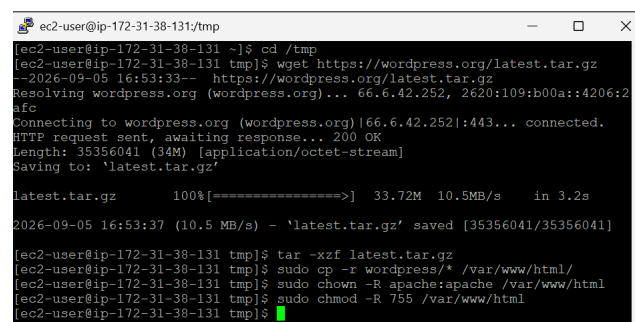
MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]> EXIT;
Bye
ec2-user@ip-172-31-38-131:~$

```

Figure 18: Setting up a local database named `wordpress_db` for the WordPress application.

I installed and extracted the WordPress package. The command `chown` made Apache the owner of the WordPress files, while `chmod` set who could read, write, or access those files. These commands ensured that Apache had the correct permissions to run WordPress.



```

ec2-user@ip-172-31-38-131:~/tmp$ cd /tmp
ec2-user@ip-172-31-38-131:~/tmp$ wget https://wordpress.org/latest.tar.gz
--2026-09-05 16:53:33-- https://wordpress.org/latest.tar.gz
Resolving wordpress.org (wordpress.org)... 66.6.42.252, 2620:109:b00a::4206:2
afc
Connecting to wordpress.org (wordpress.org)|66.6.42.252|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 35356041 (34M) [application/octet-stream]
Saving to: 'latest.tar.gz'

latest.tar.gz 100%[=====>] 33.72M 10.5MB/s in 3.2s

2026-09-05 16:53:37 (10.5 MB/s) - 'latest.tar.gz' saved [35356041/35356041]

ec2-user@ip-172-31-38-131:~/tmp$ tar -xzf latest.tar.gz
ec2-user@ip-172-31-38-131:~/tmp$ sudo cp -r wordpress/* /var/www/html/
ec2-user@ip-172-31-38-131:~/tmp$ sudo chown -R apache:apache /var/www/html
ec2-user@ip-172-31-38-131:~/tmp$ sudo chmod -R 755 /var/www/html
ec2-user@ip-172-31-38-131:~/tmp$

```

Figure 19: Installing WordPress and configuring permissions.

My WordPress application is deployed and running at `http://13.55.158.77`.

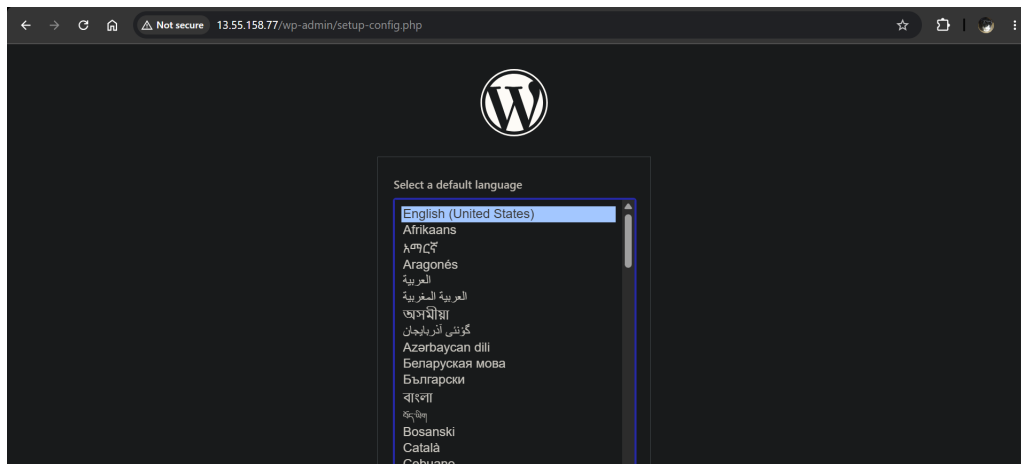


Figure 20: The deployed WordPress application.

2.2 Task 3.2

For this task, I configured my web application to use a database hosted on an Amazon RDS instance, effectively decoupling the database from the EC2 instance. I also created a backup of my web application by storing my files on an S3 bucket, and demonstrated how I could restore my application using this backup.

2.2.1 Database Decoupling

I started by creating a new RDS instance and configured it to use MariaDB as the database engine. I also connected the EC2 instance to this database right away, so that AWS automatically allowed inbound traffic on port 3306 from the web application to the database for me.

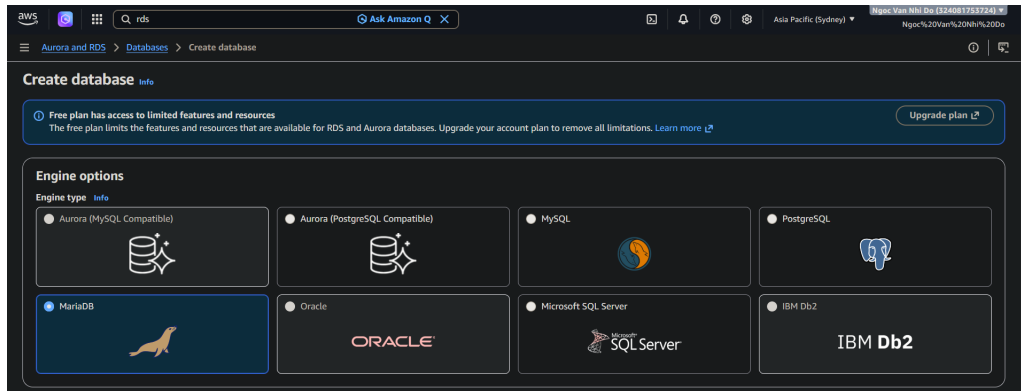


Figure 21: Creating a new RDS instance.

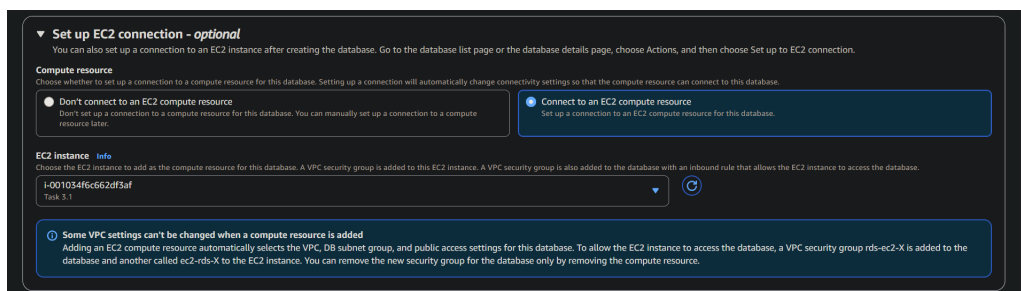


Figure 22: Connecting the EC2 instance from the previous task.

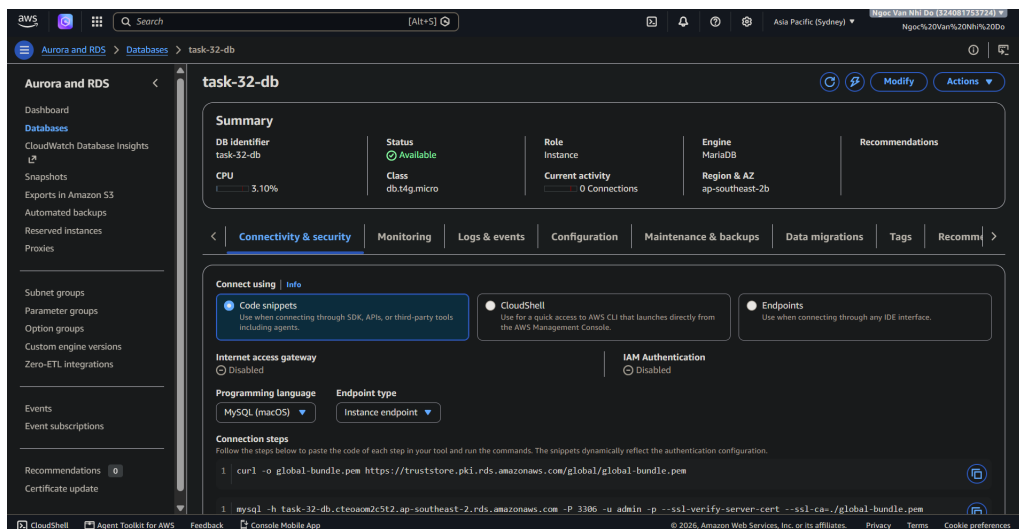


Figure 23: The database is successfully created.

From the EC2 instance's command line, I established a connection to the RDS instance using its endpoint at `task-32-db.cteoam2c5t2.ap-southeast-2.rds.amazonaws.com`.

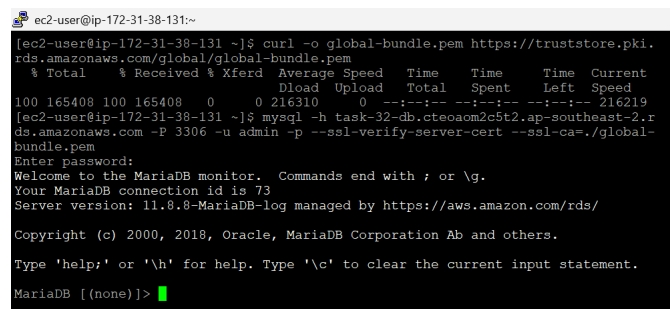


Figure 24: Connecting to the database from the EC2 instance's terminal.

Once I could connect to the database, I created a new database `wordpress_db` to be used by my application. I then exited the DB connection, configured the `[wp-config.php]` file with the required database information, including the database name, username, and password, along with the other necessary credentials. This allowed the WordPress application to connect to and use the configured database. Finally, I restarted the Apache web server and verified that the application has been successfully configured to use the database.

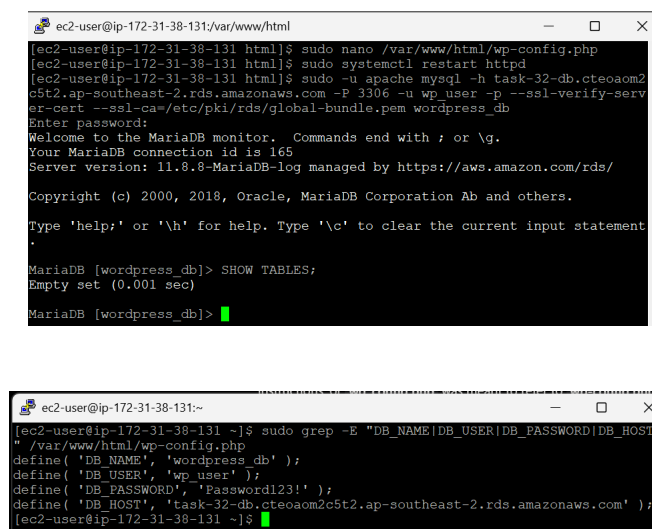


Figure 25: Editing the database configurations and verifying the connection to the RDS database.

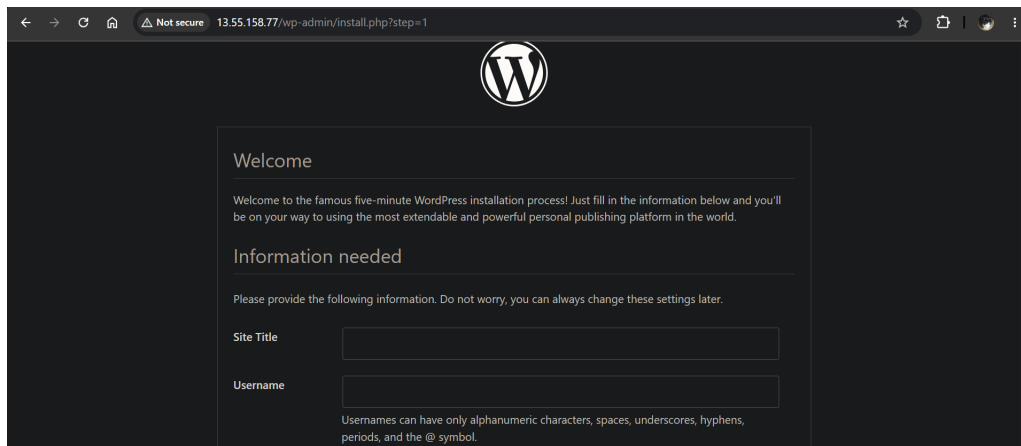


Figure 26: The application now shows the WordPress installation form, which means the database has successfully been connected.

2.2.2 Backup and Restore

I started by creating a new S3 bucket with the name `swe40006-105551765-bucket` to store my backup files.

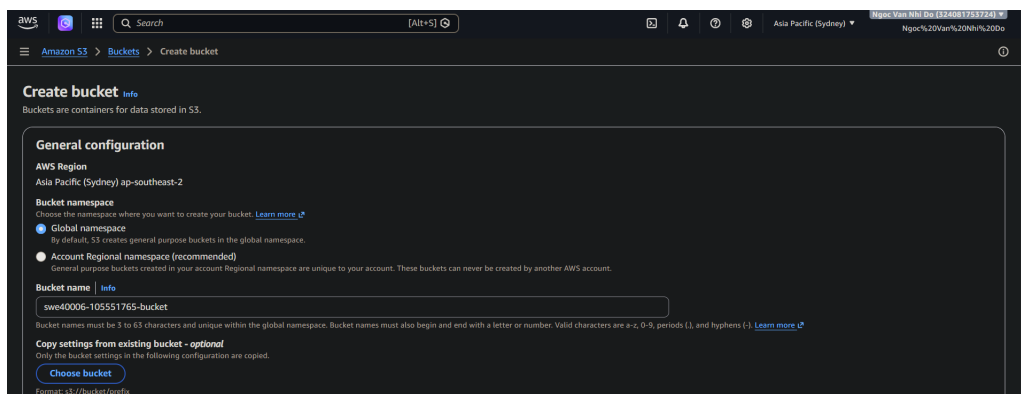


Figure 27: The application now shows the WordPress installation form, which means the database has successfully been connected.

I also created a new IAM role allowing full access to S3. This role is then attached to my EC2 instance so that the server could read and write files to the S3 bucket.

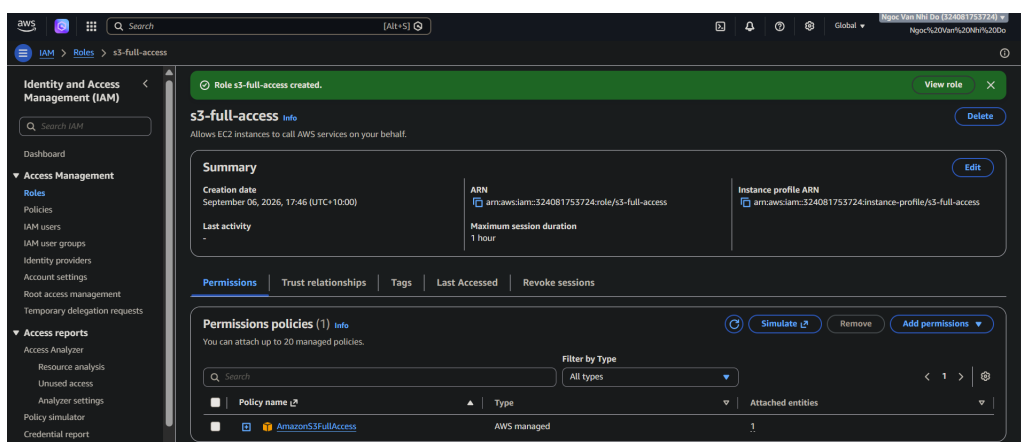


Figure 28: Creating a new IAM role with the AmazonS3FullAccess policy.



Figure 29: Adding the role to my EC2 instance.

I then made a backup of my files by archiving them into a `.tar` file to avoid having to individually handle hundreds of different files. This archive is then uploaded to the S3 bucket, serving as the backup to my web server.

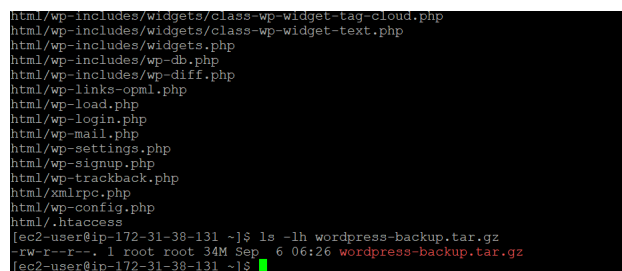
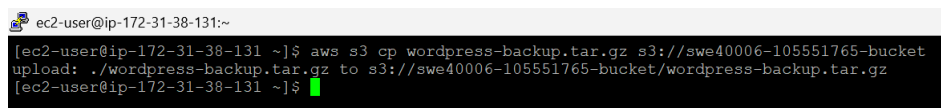
Figure 30: Creating the backup as a `.tar` archive.

Figure 31: Uploading the backup to the S3 bucket.

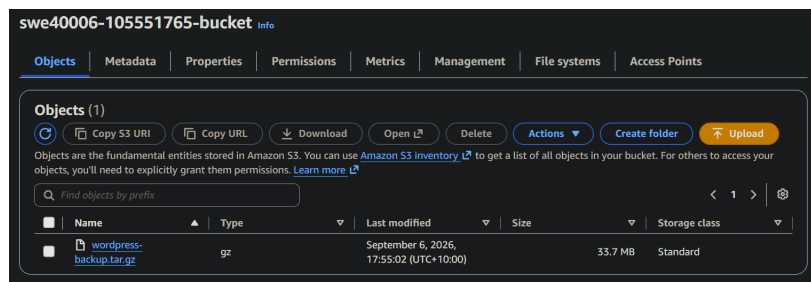


Figure 32: I could see that the backup has been uploaded using the AWS Console.

To demonstrate that I can restore my web server, I first removed all my web application files to simulate a deletion.

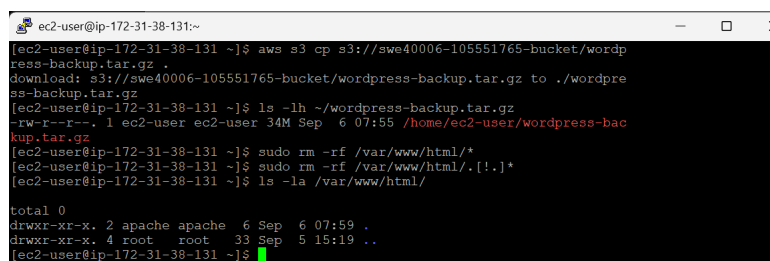


Figure 33: Removing all web application files on my EC2 instance.

Once the files were removed, accessing my website displayed only an "It works!" message rather than

the WordPress application, which is the default Apache web server page. This confirmed that the web application had been completely deleted.

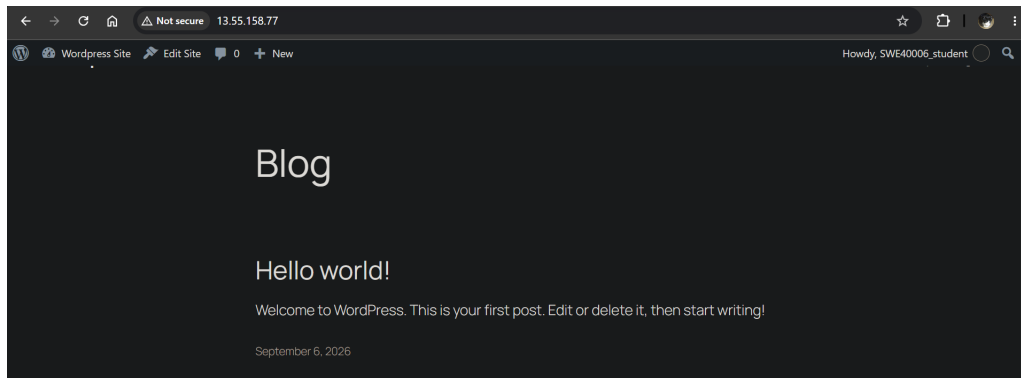


Figure 34: The WordPress application had been deleted.

To restore my deleted web application, I pulled the backup from the S3 bucket and extracted the archived files. The final step was to modify Apache's permissions to be able to read these files and restarting the web server.

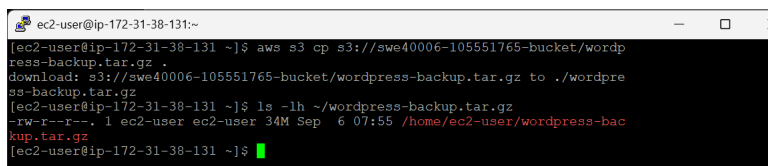


Figure 35: Downloading the backup from the S3 bucket.

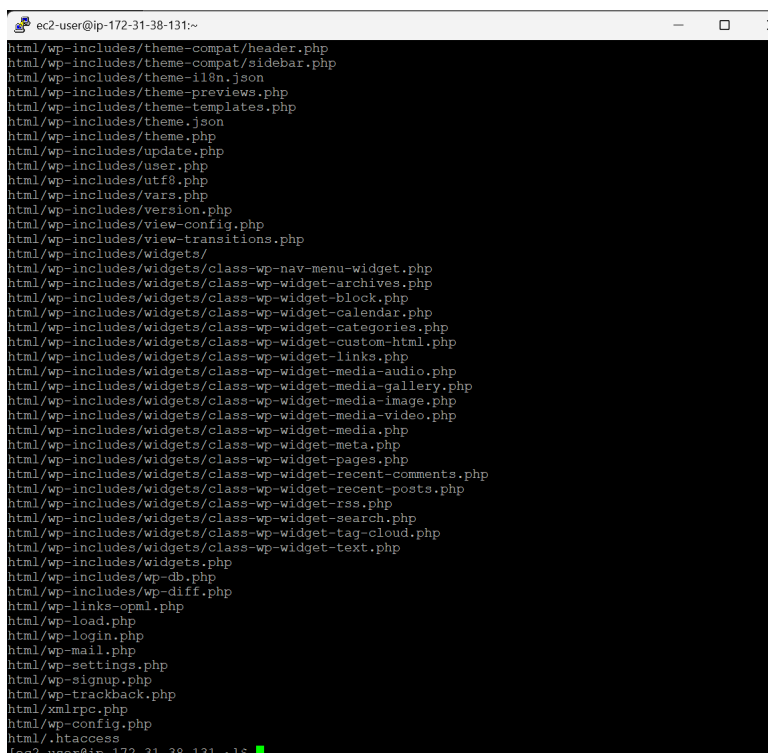


Figure 36: Extracting the web application files.

```
ec2-user@ip-172-31-38-131:~$ ls -la /var/www/html
total 272
drwxr-xr-x. 5 apache apache 16384 Sep  6 08:17 .
drwxr-xr-x. 4 root root 33 Sep  5 15:19 ..
-rw-r--r--. 1 apache apache 523 Sep  6 04:35 .htaccess
-rw-r--r--. 1 apache apache 405 Sep  5 16:54 index.php
-rw-r--r--. 1 apache apache 19903 Sep  5 16:54 license.txt
-rw-r--r--. 1 apache apache 7407 Sep  5 16:54 readme.html
-rw-r--r--. 1 apache apache 7718 Sep  5 16:54 wp-activate.php
drwxr-xr-x. 9 apache apache 16384 Sep  5 16:54 wp-admin
-rw-r--r--. 1 apache apache 351 Sep  5 16:54 wp-blog-header.php
-rw-r--r--. 1 apache apache 2323 Sep  5 16:54 wp-comments-post.php
-rw-r--r--. 1 apache apache 3339 Sep  5 16:54 wp-config-sample.php
-rw-r--r--. 1 apache apache 3427 Sep  6 03:04 wp-config.php
drwxr-xr-x. 5 apache apache 67 Sep  6 04:35 wp-content
-rw-r--r--. 1 apache apache 5617 Sep  5 16:54 wp-cron.php
drwxr-xr-x. 34 apache apache 16384 Sep  5 16:54 wp-includes
-rw-r--r--. 1 apache apache 2493 Sep  5 16:54 wp-links-opml.php
-rw-r--r--. 1 apache apache 3937 Sep  5 16:54 wp-load.php
-rw-r--r--. 1 apache apache 52536 Sep  5 16:54 wp-login.php
-rw-r--r--. 1 apache apache 8727 Sep  5 16:54 wp-mail.php
-rw-r--r--. 1 apache apache 33152 Sep  5 16:54 wp-settings.php
-rw-r--r--. 1 apache apache 35081 Sep  5 16:54 wp-signup.php
-rw-r--r--. 1 apache apache 5396 Sep  5 16:54 wp-trackback.php
-rw-r--r--. 1 apache apache 3205 Sep  5 16:54 xmlrpc.php
[ec2-user@ip-172-31-38-131 ~]$ sudo chown -R apache:apache /var/www/html
[ec2-user@ip-172-31-38-131 ~]$ sudo systemctl start httpd
[ec2-user@ip-172-31-38-131 ~]$
```

Figure 37: Verifying that the application files have been extracted.

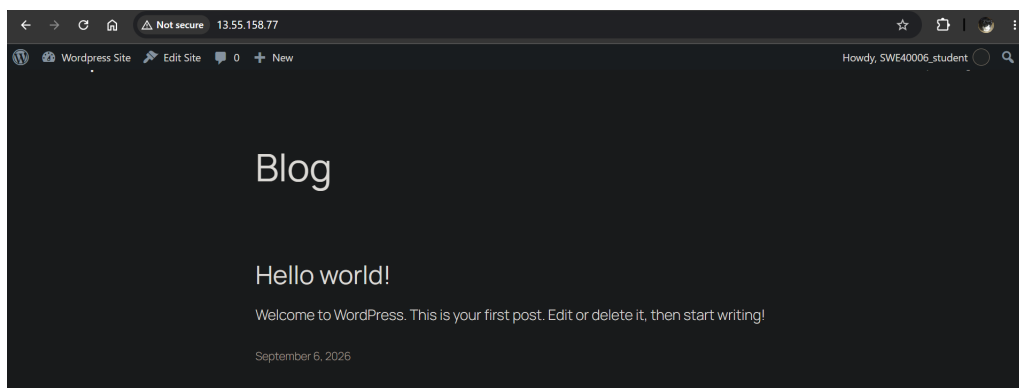


Figure 38: The web application was successfully restored.